

# Module 1: What AI Changes

And what it does not change

## Module 1: What AI Changes

COMP 331: AI-Assisted Software Engineering

Primary sources: Osmani, Chapter 1; Thomas & Hunt, Chapters 1-2 (Topics 1-15).

---

### Learning Outcomes

- Explain the difference between vibe coding and AI-assisted software engineering.
  - Describe programming with intent.
  - Summarize strengths and limits of AI-generated code.
  - Interpret Software 2.0 as a shift in specification and verification.
- 

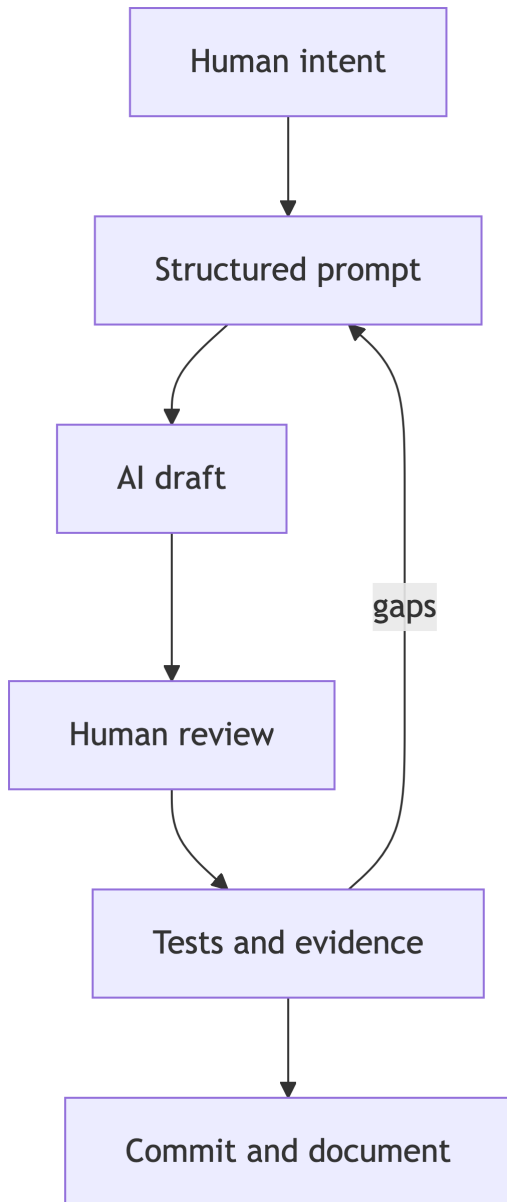
### The Core Shift

AI changes the interface to software creation.

It does not remove the need for engineering judgment.

- Before: translate intent into code directly.
  - Now: translate intent into prompts, examples, constraints, tests, reviews, and code.
  - The human role moves upstream and downstream: specify better, verify harder.
-

## AI-Assisted SWE Loop



## Vibe Coding vs Engineering

---

| Vibe Coding                | AI-Assisted Engineering            |
|----------------------------|------------------------------------|
| Conversational exploration | Structured collaboration           |
| Accepts plausible output   | Verifies behavior and risks        |
| Optimizes for momentum     | Optimizes for trusted progress     |
| Weak artifact trail        | Prompts, tests, decisions, commits |

---

---

## Programming With Intent

Programming with intent means making the desired behavior explicit before accepting implementation.

- What should the system do?
  - What should it refuse to do?
  - What edge cases matter?
  - What evidence proves it works?
  - What human decision remains non-delegable?
- 

## Pragmatic Foundation

Thomas and Hunt start with a philosophy: pragmatic programmers take responsibility, keep learning, communicate clearly, and adapt to change.

For AI-assisted SWE, this means:

- Own every generated artifact.
  - Communicate precisely with humans and tools.
  - Treat AI output as a draft, not authority.
  - Keep improving your knowledge portfolio.
-

## Pragmatic Approach

The second reading turns philosophy into design habits.

- Good design is easier to change.
  - DRY: keep each piece of knowledge in one authoritative place.
  - Orthogonality: avoid coupling unrelated decisions.
  - Reversibility: avoid choices that trap the project.
  - Tracer bullets and prototypes help test understanding early.
- 

## AI Does Not Replace Design Judgment

AI can generate code quickly, but it cannot know which future changes matter unless we supply context.

Pragmatic questions still apply:

- Is this easier to change?
  - Did the AI duplicate knowledge or intent?
  - Are components coupled for no good reason?
  - Is this a reversible experiment or a commitment?
  - What estimate or prototype would reduce uncertainty?
- 

## Where AI Helps

- Drafting boilerplate and examples.
  - Exploring alternate implementations.
  - Explaining unfamiliar APIs.
  - Creating first-pass tests and docs.
  - Speeding up setup and debugging paths.
-

## Where AI Still Struggles

- Hidden requirements and domain context.
  - Architecture tradeoffs across time.
  - Security-sensitive boundary decisions.
  - Correctness when tests are weak.
  - Knowing when the task is underspecified.
- 

## Software 2.0 Lens

Software 2.0 reframes some development as training, prompting, selecting, and evaluating behavior.

In this course, we apply that lens pragmatically:

- Prompts become one specification artifact alongside requirements, tests, and code.
  - Code review includes model-output review.
  - Engineering evidence matters more than confidence.
- 

## Lab 1: Establishing the Workflow

Assignment 1 is not just tool setup.

It establishes the course habit:

- Capture the prompt.
  - Inspect the suggestion.
  - Make a manual decision.
  - Verify the result.
  - Document what happened.
-

## Practice Activity

Ask Copilot for a small Java method.

Then improve the interaction:

- Add input constraints.
  - Add expected behavior.
  - Add one edge case.
  - Ask for a JUnit test.
  - Record what changed between attempts.
- 

## Takeaways

- AI assistance amplifies both good and bad engineering habits.
- Intent, verification, and responsibility remain human work.
- The course evidence trail is part of the engineering process.